GO PROGRAMMING HANDBOOK FOR BEGINNERS

From Zero to Building Backend Applications with Go

Part 3 of 3 — Concurrency, Systems & Real APIs

Chapters 17 – 22: Goroutines through Building REST APIs

Table of Contents

Chapter 17 — Goroutines

17.1 Concurrency vs. Parallelism

These two words are often confused. Concurrency means structuring a program so that multiple tasks can make progress independently of each other — they don't have to run at the exact same instant, just in an interleaved, non-blocking way. Parallelism means multiple tasks actually running at the same physical instant, on different CPU cores. A concurrent program can run on a single core (switching between tasks) or take advantage of parallelism on multiple cores; Go supports both, and it is the Go runtime scheduler that decides how to distribute work across the available cores.

```
  Concurrency (one core, interleaved):
  Task A: ----   ----   ----
  Task B:     ----   ----   ----

  Parallelism (two cores, simultaneous):
  Core 1 (Task A): --------------------
  Core 2 (Task B): --------------------
```

17.2 What Is a Goroutine?

A goroutine is a lightweight, independently executing function managed by the Go runtime rather than the operating system directly. Goroutines are far cheaper than operating-system threads — a Go program can comfortably run hundreds of thousands of goroutines, whereas the same number of OS threads would overwhelm most machines.

17.3 The go Keyword

Placing go before a function call runs that call as a new goroutine, and execution continues immediately to the next line without waiting for it to finish.

```
func sayHello() {
      fmt.Println("Hello from a goroutine!")
}

func main() {
      go sayHello()
      fmt.Println("Hello from main!")
      time.Sleep(100 * time.Millisecond)  // give the goroutine time to run
}
```

WARNING

main() does not wait for goroutines to finish on its own. If main() returns before a goroutine completes, that goroutine is simply abandoned. Relying on time.Sleep (as above) to "wait" is fragile and only used here for illustration — real code uses sync.WaitGroup or channels instead.

17.4 The Go Scheduler (a Brief Mental Model)

The Go runtime includes its own scheduler that multiplexes many goroutines onto a small number of OS threads, which in turn run on the available CPU cores. This scheduler automatically pauses a goroutine that is blocked (for example, waiting on a channel or I/O) and runs a different, ready goroutine in its place, so a few blocked goroutines don't waste an entire OS thread.

17.5 sync.WaitGroup

A WaitGroup lets the main goroutine wait for a known number of other goroutines to finish before continuing.

```
import (
	"fmt"
	"sync"
)

func worker(id int, wg *sync.WaitGroup) {
	defer wg.Done()          // signal completion when this function returns
	fmt.Printf("Worker %d starting\n", id)
	fmt.Printf("Worker %d done\n", id)
}

func main() {
	var wg sync.WaitGroup
	for i := 1; i <= 3; i++ {
		wg.Add(1)              // register one goroutine to wait for
		go worker(i, &wg)
	}
	wg.Wait()                  // block here until all three call Done()
	fmt.Println("All workers finished")
}
```

17.6 Race Conditions and sync.Mutex

A race condition happens when two or more goroutines access the same variable at the same time, and at least one of them writes to it, producing unpredictable results. A sync.Mutex ("mutual exclusion lock") protects shared data by allowing only one goroutine at a time to access the code between Lock() and Unlock().

```
type Counter struct {
	mu    sync.Mutex
	value int
}

func (c *Counter) Increment() {
	c.mu.Lock()
	defer c.mu.Unlock()
	c.value++
```

```
}
```

TIP

Run "go run -race main.go" during development. Go's built-in race detector instruments your program and reports data races it observes while running, which is invaluable for catching concurrency bugs early.

17.7 A Practical Example: Concurrent Downloads (Simulated)

```
func fetch(url string, wg *sync.WaitGroup) {
	defer wg.Done()
	time.Sleep(200 * time.Millisecond) // simulate network delay
	fmt.Println("fetched:", url)
}

func main() {
	urls := []string{"a.com", "b.com", "c.com"}
	var wg sync.WaitGroup
	for _, u := range urls {
		wg.Add(1)
		go fetch(u, &wg)
	}
	wg.Wait()
	fmt.Println("All fetches complete")
	// All three "fetches" run concurrently, finishing in roughly 200ms total,
	// rather than roughly 600ms if done one after another.
}
```

Chapter Summary

- Concurrency is about structuring independent tasks; parallelism is about running tasks at the same physical instant.
- The go keyword launches a function as a lightweight goroutine, managed by Go's own runtime scheduler.
- sync.WaitGroup lets a goroutine wait for a known number of other goroutines to finish.
- sync.Mutex prevents race conditions by allowing only one goroutine at a time into a protected section of code.
- "go run -race" is an essential tool for catching concurrency bugs during development.

Practice Exercises

Exercise 1: Write a function that launches 5 goroutines, each printing its own number (1 through 5), and use a WaitGroup to wait for all of them.

Exercise 2: Explain, in your own words, why two goroutines incrementing the same int variable without a mutex can produce an incorrect final total.

Exercise 3: What command-line flag helps you detect race conditions while testing a Go program?

Solutions

Solution:

```
var wg sync.WaitGroup
for i := 1; i <= 5; i++ {
	wg.Add(1)
	go func(n int) {
		defer wg.Done()
		fmt.Println(n)
	}(i)
}
wg.Wait()
```

2. Incrementing a variable (like count++) is not a single atomic step — it involves reading the current value, adding one, and writing the result back. If two goroutines both read the same current value before either has written its update back, one goroutine's increment can be silently overwritten by the other's, so the final total ends up lower than the correct count. A sync.Mutex (or an atomic operation) prevents this by ensuring only one goroutine performs the whole read-modify-write sequence at a time.

3. The "-race" flag, used as "go run -race main.go" or "go test -race", enables Go's built-in race detector.

Chapter 18 — Channels

18.1 What Is a Channel?

A channel is a typed conduit through which goroutines send and receive values, letting them communicate and synchronize safely without a mutex. Go's design philosophy is famously summarized as: "Do not communicate by sharing memory; instead, share memory by communicating." Channels are the mechanism behind that idea.

```
  goroutine A  --value-->  [ channel ]  --value-->  goroutine B
```

18.2 Creating Channels

```
ch := make(chan int)        // an unbuffered channel of int
ch2 := make(chan string, 5) // a buffered channel of string, capacity 5
```

18.3 Sending and Receiving

```
func main() {
	ch := make(chan string)

	go func() {
		ch <- "hello from the goroutine"  // send
	}()

	message := <-ch    // receive (blocks until a value arrives)
	fmt.Println(message)
}
```

An unbuffered channel's send blocks until another goroutine is ready to receive, and vice versa — this handoff is itself a synchronization point, which is why the example above works correctly without a WaitGroup.

18.4 Buffered Channels

A buffered channel can hold a limited number of values without a receiver being ready yet. A send only blocks once the buffer is full; a receive only blocks once the buffer is empty.

```
ch := make(chan int, 2)
ch <- 1   // does not block: buffer has room
ch <- 2   // does not block: buffer now full
// ch <- 3   would block here until something is received
fmt.Println(<-ch)  // 1
fmt.Println(<-ch)  // 2
```

18.5 Closing Channels and range

A sender can close(ch) to signal that no more values will be sent. Receivers can detect this using the two-value receive form, and range over a channel automatically stops once it is closed.

```
func generate(ch chan int) {
	for i := 1; i <= 3; i++ {
		ch <- i
	}
	close(ch)
}

func main() {
	ch := make(chan int)
	go generate(ch)
	for value := range ch {  // exits automatically once ch is closed
		fmt.Println(value)
	}
}
```

WARNING

Only the sender should close a channel, and never more than once — closing an already-closed channel, or sending to a closed channel, causes a panic. Receiving from a closed channel is safe and immediately returns the zero value.

18.6 select

select lets a goroutine wait on multiple channel operations at once, proceeding with whichever one becomes ready first.

```
func main() {
	ch1 := make(chan string)
	ch2 := make(chan string)

	go func() { time.Sleep(50 * time.Millisecond); ch1 <- "from ch1" }()
	go func() { time.Sleep(20 * time.Millisecond); ch2 <- "from ch2" }()

	select {
	case msg1 := <-ch1:
		fmt.Println(msg1)
	case msg2 := <-ch2:
		fmt.Println(msg2)   // this one prints, since ch2 is faster
	}
}
```

select can also include a default case, which runs immediately if no channel is ready, letting you perform a non-blocking check.

18.7 Deadlocks

A deadlock occurs when a goroutine is blocked waiting for something that will never happen — commonly, an unbuffered channel send or receive with no other goroutine on the other end. Go's runtime detects the simplest case (the entire program deadlocked) and terminates with a clear error rather than hanging forever silently.

```
func main() {
	ch := make(chan int)
	ch <- 1   // blocks forever: no other goroutine will ever receive
	// fatal error: all goroutines are asleep - deadlock!
}
```

18.8 Worker Pool Pattern

```
func worker(id int, jobs <-chan int, results chan<- int) {
	for j := range jobs {
		results <- j * 2
	}
}

func main() {
	jobs := make(chan int, 5)
	results := make(chan int, 5)

	for w := 1; w <= 3; w++ {
		go worker(w, jobs, results)
	}
	for j := 1; j <= 5; j++ {
		jobs <- j
	}
	close(jobs)

	for a := 1; a <= 5; a++ {
		fmt.Println(<-results)
	}
}
```

The parameter types <-chan int (receive-only) and chan<- int (send-only) document each function's intent and let the compiler catch misuse — for instance, a worker cannot accidentally send on jobs.

Chapter Summary

- Channels let goroutines communicate and synchronize by sending and receiving typed values.
- Unbuffered channels synchronize sender and receiver directly; buffered channels tolerate a limited backlog.
- close(ch) signals no more values are coming; range over a channel stops automatically when it closes.
- select waits on multiple channel operations, proceeding with whichever is ready first.

- A deadlock happens when goroutines wait on each other (or on channels) with no way forward; Go detects whole-program deadlocks and reports them.

Practice Exercises

Exercise 1: Write a program that sends the numbers 1 to 5 into a channel from a goroutine, then prints each one using range in main.

Exercise 2: What is the difference between an unbuffered channel and a buffered channel of capacity 3?

Exercise 3: Why does sending to a channel with no receiver, and no buffer space, cause a deadlock?

Solutions

Solution:

```
func main() {
	ch := make(chan int)
	go func() {
		for i := 1; i <= 5; i++ {
			ch <- i
		}
		close(ch)
	}()
	for v := range ch {
		fmt.Println(v)
	}
}
```

2. An unbuffered channel has no internal storage: a send blocks until a receiver is ready at that exact moment, and vice versa — it is a direct handoff. A buffered channel of capacity 3 can hold up to 3 values with no receiver waiting; a send only blocks once those 3 slots are full, and a receive only blocks once the buffer is empty.

3. A channel send only completes once there is somewhere for the value to go — either a waiting receiver (for an unbuffered channel) or free buffer space (for a buffered channel). If neither exists and never will, the sending goroutine remains blocked indefinitely with no way to make progress, which is exactly what a deadlock is.

Chapter 19 — Packages and Modules

19.1 What Is a Package?

A package is Go's unit of code organization: a directory of .go files that all declare the same package name and are compiled together. Every Go file belongs to exactly one package, declared on its very first line.

```
package mathutil

func Add(a, b int) int {
	return a + b
}
```

19.2 Importing Packages

```
import (
	"fmt"                        // standard library package
	"myproject/internal/mathutil" // a package from your own module
)

func main() {
	fmt.Println(mathutil.Add(2, 3))
}
```

Only exported names — those starting with a capital letter — are visible outside the package they're declared in. mathutil.Add works because Add is capitalized; mathutil.add would not compile from outside the package.

19.3 What Is a Module?

A module is one or more packages released and versioned together, identified by a module path (often matching a repository URL, like github.com/username/project) and described by a go.mod file at its root.

```
$ go mod init github.com/username/myproject

module github.com/username/myproject

go 1.22
```

19.4 go.mod and go.sum

- go.mod — declares the module's own path, the Go version it targets, and the exact versions of any external dependencies it requires.

- go.sum — records cryptographic checksums of every dependency (and its dependencies), so builds are reproducible and tampering can be detected.

Adding a third-party dependency is as simple as importing it in code and running:

```
$ go get github.com/google/uuid
```

This downloads the package, adds a require line to go.mod, and updates go.sum automatically.

19.5 Organizing a Real Project

```
myproject/
├── go.mod
├── go.sum
├── main.go               (package main)
├── internal/
│   ├── handlers/         (HTTP handlers — package handlers)
│   ├── models/           (data structures — package models)
│   └── storage/          (database access — package storage)
└── pkg/
    └── validator/        (reusable, importable by other modules)
```

- internal/ is enforced by the Go compiler: packages under it can only be imported by code inside the same module tree, making it ideal for implementation details you don't want other projects depending on.
- pkg/ (a convention, not enforced by the compiler) signals code that is intended to be reusable and imported by other projects.

19.6 Naming Conventions

- Package names are short, lowercase, single words — http, json, mathutil, not math_util or MathUtil.
- Avoid stuttering: in package models, name a type User, not ModelsUser — callers already write models.User.
- Avoid a generic package named utils or common when possible; a more specific name (e.g. stringutil) documents intent better.

19.7 Versioning and Semantic Versioning

Go modules follow semantic versioning: vMAJOR.MINOR.PATCH (e.g. v1.4.2). Increasing MAJOR signals breaking changes, MINOR signals backward-compatible new features, and PATCH signals backward-compatible bug fixes. From v2 onward, a module's import path must include the major version (e.g. github.com/user/project/v2), which is a deliberate Go convention to let multiple incompatible major versions coexist in a dependency graph.

Chapter Summary

- A package groups related source files under one package name; only capitalized identifiers are exported outside it.
- A module is a versioned collection of packages, defined by a go.mod file at its root.
- go.sum locks dependency checksums for reproducible, tamper-evident builds.
- internal/ restricts imports to within the same module; pkg/ conventionally marks reusable code.
- Go modules follow semantic versioning, with major-version-2-and-above baked into the import path.

Practice Exercises

Exercise 1: What command initializes a new module named github.com/alex/todoapp?

Exercise 2: Why can code in another module not import anything from a package under your project's internal/ directory?

Exercise 3: Given a package named stringutil with a function "func Reverse(s string) string", write the import and call needed to use it from package main.

Solutions

Solution:

```
$ go mod init github.com/alex/todoapp
```

2. The Go toolchain specifically recognizes any directory named internal (at any depth) and refuses, at compile time, to let code outside the module rooted just above that internal directory import packages inside it. This is a language-level convention, not just a naming suggestion, so it's guaranteed to be enforced regardless of who is building the code.

```
import "myproject/stringutil"

result := stringutil.Reverse("hello")
```

Chapter 20 — File Handling

20.1 Reading an Entire File

For small-to-medium files, os.ReadFile is the simplest way to read a file's entire contents into memory as a []byte.

```
import (
	"fmt"
	"os"
)

func main() {
	data, err := os.ReadFile("notes.txt")
	if err != nil {
		fmt.Println("Error reading file:", err)
		return
	}
	fmt.Println(string(data))
}
```

20.2 Writing a File

```
content := []byte("Hello, file!")
err := os.WriteFile("output.txt", content, 0644)
if err != nil {
	fmt.Println("Error writing file:", err)
}
```

The final argument, 0644, is a Unix file permission: the owner can read and write, while everyone else can only read.

20.3 Reading Large Files Line by Line

For large files, reading everything into memory at once is wasteful. bufio.Scanner reads a file incrementally, one line at a time.

```
import (
	"bufio"
	"fmt"
	"os"
)

func main() {
	file, err := os.Open("large.txt")
	if err != nil {
		fmt.Println("Error:", err)
		return
```

```
	}
	defer file.Close()

	scanner := bufio.NewScanner(file)
	for scanner.Scan() {
		fmt.Println(scanner.Text())
	}
}
```

TIP

defer file.Close() immediately after a successful Open schedules cleanup to run when the enclosing function returns, guaranteeing the file handle is released even if you return early or the function later panics.

20.4 Working with Directories

```
os.Mkdir("reports", 0755)             // create one directory
os.MkdirAll("data/2026/07", 0755)     // create nested directories

entries, err := os.ReadDir(".")
if err == nil {
	for _, entry := range entries {
		fmt.Println(entry.Name(), entry.IsDir())
	}
}
```

20.5 Working with JSON Files

Go's encoding/json package converts between Go values and JSON text. Struct tags control the JSON field names.

```
type Config struct {
	Name    string `json:"name"`
	Version int    `json:"version"`
}

func main() {
	cfg := Config{Name: "MyApp", Version: 2}

	// Struct -> JSON
	jsonBytes, _ := json.MarshalIndent(cfg, "", "  ")
	os.WriteFile("config.json", jsonBytes, 0644)

	// JSON -> Struct
	data, _ := os.ReadFile("config.json")
	var loaded Config
	json.Unmarshal(data, &loaded)
	fmt.Println(loaded.Name, loaded.Version)
}
```

20.6 Working with CSV Files

The encoding/csv package reads and writes comma-separated value files, a common format for spreadsheets and data exports.

```
import (
	"encoding/csv"
	"os"
)

func main() {
	file, _ := os.Open("people.csv")
	defer file.Close()

	reader := csv.NewReader(file)
	records, err := reader.ReadAll()
	if err != nil {
		return
	}
	for _, row := range records {
		fmt.Println(row) // row is []string, e.g. ["Ada" "28" "Lagos"]
	}
}
```

20.7 Real-World Example: Logging User Activity to a File

```
func logActivity(message string) error {
	file, err := os.OpenFile("activity.log",
os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
	if err != nil {
		return err
	}
	defer file.Close()

	_, err = file.WriteString(time.Now().Format(time.RFC3339) + " " + message +
"\n")
	return err
}
```

os.O_APPEND ensures new writes are added to the end of the file rather than overwriting it; os.O_CREATE creates the file if it doesn't already exist.

Chapter Summary

- os.ReadFile and os.WriteFile handle whole-file reads and writes for small-to-medium files.
- bufio.Scanner reads large files efficiently, one line at a time.
- defer file.Close() right after a successful Open guarantees the file handle is released.
- encoding/json and encoding/csv convert between Go values and the two most common data-interchange file formats.

- os.OpenFile with flags like O_APPEND and O_CREATE gives fine control over how a file is opened.

Practice Exercises

Exercise 1: Write a program that writes the text "Learning Go" to a file called "progress.txt".

Exercise 2: Write a program that reads "progress.txt" back and prints its contents.

Exercise 3: Why is bufio.Scanner generally preferable to os.ReadFile for a multi-gigabyte log file?

Solutions

Solution:

```
err := os.WriteFile("progress.txt", []byte("Learning Go"), 0644)
if err != nil {
	fmt.Println("Error:", err)
}

data, err := os.ReadFile("progress.txt")
if err != nil {
	fmt.Println("Error:", err)
	return
}
fmt.Println(string(data))
```

3. os.ReadFile loads the entire file into memory at once, so a multi-gigabyte file would require a multi-gigabyte allocation, which can exhaust available memory or slow the program dramatically. bufio.Scanner instead reads and processes the file incrementally, one line (or token) at a time, keeping memory usage small and constant regardless of the total file size.

Chapter 21 — Error Handling

21.1 The error Interface

Go has no exceptions for ordinary error handling. Instead, functions that can fail return an extra value of the built-in error interface type, which any type can satisfy by implementing a single method:

```
type error interface {
	Error() string
}
```

By convention, error is always the last return value, and nil means "no error occurred."

```
func divide(a, b float64) (float64, error) {
	if b == 0 {
		return 0, errors.New("cannot divide by zero")
	}
	return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
	fmt.Println("Error:", err)
	return
}
fmt.Println("Result:", result)
```

TIP

The Go community's convention is: "check errors immediately after the call that might produce one." Reading Go code, you'll see "if err != nil" constantly — this repetition is intentional, keeping control flow explicit and visible rather than hidden in a try/catch elsewhere.

21.2 Creating Errors: errors.New and fmt.Errorf

```
err1 := errors.New("file not found")
err2 := fmt.Errorf("failed to process user %d: %w", userID, err1)  // %w wraps
err1
```

fmt.Errorf's %w verb wraps an existing error inside a new one, preserving the original for later inspection while adding context.

21.3 Custom Error Types

Defining your own error type lets callers extract structured details, not just a message string.

```
type ValidationError struct {
	Field   string
	Message string
```

```
}

func (e *ValidationError) Error() string {
	return fmt.Sprintf("%s: %s", e.Field, e.Message)
}

func validateAge(age int) error {
	if age < 0 {
		return &ValidationError{Field: "age", Message: "cannot be negative"}
	}
	return nil
}
```

21.4 errors.Is and errors.As

When errors are wrapped, errors.Is checks whether a specific sentinel error appears anywhere in the chain, and errors.As extracts a specific error type from the chain.

```
var ErrNotFound = errors.New("not found")

func findUser(id int) error {
	return fmt.Errorf("lookup failed: %w", ErrNotFound)
}

err := findUser(42)
if errors.Is(err, ErrNotFound) {
	fmt.Println("The user genuinely does not exist.")
}

var ve *ValidationError
if errors.As(err, &ve) {
	fmt.Println("Field with problem:", ve.Field)
}
```

21.5 panic and recover

panic stops normal execution immediately and begins unwinding the call stack, running any deferred functions along the way. recover, called inside a deferred function, stops a panic in progress and lets the program continue. panic/recover is reserved for truly exceptional, unrecoverable-by-normal-means situations — not a substitute for ordinary error returns.

```
func safeDivide(a, b int) (result int, err error) {
	defer func() {
		if r := recover(); r != nil {
			err = fmt.Errorf("recovered from panic: %v", r)
		}
	}()
	result = a / b   // dividing by zero here panics with a run-time error
	return
```

```
}

result, err := safeDivide(10, 0)
if err != nil {
	fmt.Println(err)   // recovered from panic: runtime error: integer divide
by zero
}
```

WARNING

Do not use panic/recover as a general control-flow mechanism for expected failure cases (like "user not found" or "invalid input"). Reserve it for programmer errors and truly unrecoverable states; use ordinary returned error values for everything else.

21.6 Best Practices

- Check every error immediately; never silently discard one with "_" unless you have a specific, documented reason.
- Add context when propagating an error upward (fmt.Errorf("...: %w", err)), so a failure deep in the call stack is still understandable at the top.
- Reserve panic for programmer bugs (like nil-pointer dereferences you didn't expect) or truly unrecoverable states, not routine failures.
- Define sentinel errors (var ErrX = errors.New(...)) for conditions callers may need to check specifically with errors.Is.

Chapter Summary

- Go represents failure with ordinary returned error values, conventionally the last return value, checked with "if err != nil".
- errors.New and fmt.Errorf (with %w) create and wrap errors; custom types can carry structured detail.
- errors.Is and errors.As inspect a chain of wrapped errors for a sentinel value or specific type.
- panic/recover handles truly exceptional situations, not everyday error handling — recover only works inside a deferred function.

Practice Exercises

Exercise 1: Write a function "sqrt" that returns an error if given a negative number, instead of a result.

Exercise 2: Write a custom error type "OutOfStockError" with a Product field, and a function that returns one.

Exercise 3: Why is errors.Is generally safer than comparing errors directly with ==, once wrapping is involved?

Solutions

Solution:

```
func sqrt(x float64) (float64, error) {
	if x < 0 {
		return 0, fmt.Errorf("cannot take square root of negative number %v",
x)
	}
	return math.Sqrt(x), nil
}

type OutOfStockError struct {
	Product string
}

func (e *OutOfStockError) Error() string {
	return e.Product + " is out of stock"
}

func checkStock(product string, quantity int) error {
	if quantity == 0 {
		return &OutOfStockError{Product: product}
	}
	return nil
}
```

3. Once an error has been wrapped (for example, with fmt.Errorf and %w), the value returned to the caller is a new error object, not the original sentinel error — a direct == comparison against the sentinel will be false even though the sentinel is logically "inside" it. errors.Is walks the whole chain of wrapped errors, unwrapping each layer, and checks each one against the target, correctly reporting a match regardless of how many layers of wrapping occurred.

Chapter 22 — Building REST APIs

22.1 What Is a REST API?

REST (Representational State Transfer) is a style of designing web APIs around resources (like "users" or "products"), identified by URLs, and manipulated using standard HTTP methods. A client sends an HTTP request; the server processes it and returns an HTTP response, typically carrying data as JSON.

```
  Client                          Server (Go)
  GET /users/7      -------->     look up user 7
                    <--------     200 OK  { "id":7, "name":"Ada" }
```

22.2 HTTP Methods and Status Codes

- GET — retrieve a resource; should never change server state.
- POST — create a new resource.
- PUT — replace an existing resource entirely.
- DELETE — remove a resource.

Common status codes: 200 OK (success), 201 Created (a POST succeeded), 204 No Content (success, nothing to return), 400 Bad Request (invalid input), 404 Not Found (resource doesn't exist), 500 Internal Server Error (something went wrong on the server).

22.3 A Minimal HTTP Server

```
package main

import (
	"fmt"
	"net/http"
)

func homeHandler(w http.ResponseWriter, r *http.Request) {
	fmt.Fprintln(w, "Welcome to the API!")
}

func main() {
	http.HandleFunc("/", homeHandler)
	fmt.Println("Server running on http://localhost:8080")
	http.ListenAndServe(":8080", nil)
}
```

http.HandleFunc registers a function to run whenever a request matches a given path. Every handler receives an http.ResponseWriter (used to build the response) and a *http.Request (describing the incoming request).

22.4 Encoding and Decoding JSON

```
type User struct {
	ID   int    `json:"id"`
	Name string `json:"name"`
}

func getUserHandler(w http.ResponseWriter, r *http.Request) {
	user := User{ID: 1, Name: "Ada"}
	w.Header().Set("Content-Type", "application/json")
	json.NewEncoder(w).Encode(user)
}

func createUserHandler(w http.ResponseWriter, r *http.Request) {
	var newUser User
	if err := json.NewDecoder(r.Body).Decode(&newUser); err != nil {
		http.Error(w, "invalid JSON", http.StatusBadRequest)
		return
	}
	w.WriteHeader(http.StatusCreated)
	json.NewEncoder(w).Encode(newUser)
}
```

22.5 Routing with a Multiplexer

Go 1.22 significantly improved the standard library's router (http.ServeMux), adding support for HTTP-method matching and path parameters without any third-party dependency:

```
mux := http.NewServeMux()
mux.HandleFunc("GET /users/{id}", getUserHandler)
mux.HandleFunc("POST /users", createUserHandler)
mux.HandleFunc("PUT /users/{id}", updateUserHandler)
mux.HandleFunc("DELETE /users/{id}", deleteUserHandler)

http.ListenAndServe(":8080", mux)

func getUserHandler(w http.ResponseWriter, r *http.Request) {
	id := r.PathValue("id")
	fmt.Fprintln(w, "Looking up user", id)
}
```

TIP

For larger projects, popular third-party routers like chi or gorilla/mux add conveniences such as middleware chaining and sub-routers — but the standard library alone is now sufficient for many APIs.

22.6 Building a Complete In-Memory CRUD API

The following single-file example implements full Create, Read, Update, and Delete operations for a "task" resource, storing data in memory (a map protected by a mutex, since handlers may run concurrently).

```
package main

import (
	"encoding/json"
	"net/http"
	"strconv"
	"sync"
)

type Task struct {
	ID   int    `json:"id"`
	Name string `json:"name"`
	Done bool   `json:"done"`
}

var (
	mu     sync.Mutex
	tasks  = map[int]Task{}
	nextID = 1
)

func listTasks(w http.ResponseWriter, r *http.Request) {
	mu.Lock()
	defer mu.Unlock()
	result := make([]Task, 0, len(tasks))
	for _, t := range tasks {
		result = append(result, t)
	}
	json.NewEncoder(w).Encode(result)
}

func createTask(w http.ResponseWriter, r *http.Request) {
	var t Task
	if err := json.NewDecoder(r.Body).Decode(&t); err != nil {
		http.Error(w, "invalid JSON", http.StatusBadRequest)
		return
	}
	mu.Lock()
	t.ID = nextID
	nextID++
	tasks[t.ID] = t
	mu.Unlock()
	w.WriteHeader(http.StatusCreated)
	json.NewEncoder(w).Encode(t)
}

func getTask(w http.ResponseWriter, r *http.Request) {
	id, _ := strconv.Atoi(r.PathValue("id"))
```

```
	mu.Lock()
	t, ok := tasks[id]
	mu.Unlock()
	if !ok {
		http.Error(w, "task not found", http.StatusNotFound)
		return
	}
	json.NewEncoder(w).Encode(t)
}

func updateTask(w http.ResponseWriter, r *http.Request) {
	id, _ := strconv.Atoi(r.PathValue("id"))
	var updated Task
	if err := json.NewDecoder(r.Body).Decode(&updated); err != nil {
		http.Error(w, "invalid JSON", http.StatusBadRequest)
		return
	}
	mu.Lock()
	defer mu.Unlock()
	if _, ok := tasks[id]; !ok {
		http.Error(w, "task not found", http.StatusNotFound)
		return
	}
	updated.ID = id
	tasks[id] = updated
	json.NewEncoder(w).Encode(updated)
}

func deleteTask(w http.ResponseWriter, r *http.Request) {
	id, _ := strconv.Atoi(r.PathValue("id"))
	mu.Lock()
	defer mu.Unlock()
	if _, ok := tasks[id]; !ok {
		http.Error(w, "task not found", http.StatusNotFound)
		return
	}
	delete(tasks, id)
	w.WriteHeader(http.StatusNoContent)
}

func main() {
	mux := http.NewServeMux()
	mux.HandleFunc("GET /tasks", listTasks)
	mux.HandleFunc("POST /tasks", createTask)
	mux.HandleFunc("GET /tasks/{id}", getTask)
	mux.HandleFunc("PUT /tasks/{id}", updateTask)
	mux.HandleFunc("DELETE /tasks/{id}", deleteTask)

	http.ListenAndServe(":8080", mux)
}
```

22.7 Suggested Project Structure

```
taskapi/
├── go.mod
├── main.go              (starts the server, wires routes)
├── internal/
│   ├── models/task.go   (the Task struct)
│   ├── store/task_store.go  (in-memory or database storage)
│   └── handlers/task_handlers.go  (HTTP handler functions)
```

22.8 Testing the API

You can test a running server manually using curl:

```
$ curl -X POST localhost:8080/tasks -d '{"name":"Buy milk","done":false}'
$ curl localhost:8080/tasks
$ curl -X DELETE localhost:8080/tasks/1
```

For automated tests, Go's httptest package lets you exercise handlers directly, without starting a real network server:

```
import (
	"net/http"
	"net/http/httptest"
	"testing"
)

func TestListTasks(t *testing.T) {
	req := httptest.NewRequest("GET", "/tasks", nil)
	w := httptest.NewRecorder()

	listTasks(w, req)

	if w.Code != http.StatusOK {
		t.Errorf("expected status 200, got %d", w.Code)
	}
}
```

Run tests with go test ./..., which discovers and runs every _test.go file in the module.

Chapter Summary

- REST APIs expose resources at URLs, manipulated through HTTP methods (GET, POST, PUT, DELETE) and status codes.
- net/http provides everything needed for a working server; encoding/json handles request and response bodies.
- Go 1.22's http.ServeMux supports method-specific routes and path parameters (r.PathValue) without external dependencies.
- A mutex protects shared in-memory data structures accessed concurrently by multiple request handlers.
- httptest lets you test handlers directly and quickly, without a real network connection.

Practice Exercises

Exercise 1: Add a GET /tasks/{id}/status endpoint that returns just the "done" field of a task as JSON.

Exercise 2: What status code should a successful DELETE typically return, and why not 200?

Exercise 3: Why does the CRUD example use a sync.Mutex around the tasks map?

Solutions

Solution:

```
func getTaskStatus(w http.ResponseWriter, r *http.Request) {
	id, _ := strconv.Atoi(r.PathValue("id"))
	mu.Lock()
	t, ok := tasks[id]
	mu.Unlock()
	if !ok {
		http.Error(w, "task not found", http.StatusNotFound)
		return
	}
	json.NewEncoder(w).Encode(map[string]bool{"done": t.Done})
}
```

2. 204 No Content is the conventional choice, because a successful delete leaves nothing meaningful to return in the response body — the resource is gone. Returning 200 OK would technically work but implies there is response content worth reading, which there usually isn't for a delete.

3. Each incoming HTTP request in Go's net/http server is handled in its own goroutine, so multiple requests can read and write the shared "tasks" map at the same time. Without a mutex, concurrent map access in Go is a race condition that can corrupt the map's internal structure and crash the program; sync.Mutex ensures only one goroutine touches the map at a time.